Unit 08: Storage Classes

CONTENTS

Objectives

After studying this unit, you will be able to:

- storage classes
- Scope of a variable
- Auto, Static, Extern and Register

Introduction

Storage Classes are used to characterize a variable's or function's characteristics. These characteristics include scope, visibility, and life-time, which allow us to track the presence of a variable through the course of a program's execution. The following items are described by a storage class in C:

The variable scope.

The location where the variable will be stored.

The initialized value of a variable.

A lifetime of a variable.

Who can access a variable?

8.1 Storage Classes and their Usage

There are two different ways to characterize variables:

1. by data types
2. by storage class

Data types refers to the type of information while storage class refers to the life-time of a variable and its scope within the program.

A variable in C can have any one of the four storage classes.

1. Automatic variable
2. External variable
3. Static variable
4. Register variable

8.2 Automatic Variable

An automatic variable's scope is limited to the function in which it is declared. When the function is called, it is formed, and when the function is exited, it is automatically deleted. As a result, the name Automatic was chosen.

Local variables are variables defined with the auto storage class. The term "auto" refers to the automatic storage class. If a variable is not explicitly declared, it is in the auto storage class by default.

The scope of an auto variable is restricted to a single block. The access is destroyed once the control leaves the block. This means that the auto variable can only be accessed from the block in which it is declared.

A keyword auto is used to define an auto storage class. By default, an auto variable contains a garbage value.

By default, a variable declared inside a function with storage class specification is an automatic variable. Automatic variable values cannot be changed accidently by what happens in some other functions in the program.

```
main( )
{
int m = 1000;
function 2( );
printf (“ %d \n”, m);
}
function 1( )
{
int m = 10;
printf (“ %d \n”, m);
}
function 2( )
{
int m = 100;
function 1( );
printf (“ %d \n”, m);
}
output: 10
100
1000
```

8.3 External Variable

An external variable is also known as a global variable. It is not confined to a single function. Its scope extends from the point of definition through the remainder of the program.

External variables can be accessed from any function that falls within their scope. They are declared outside a function. If a local variable and a global variable have the same name, local variable will have precedence over global in the function where it is declared.

```
int count; main
{
count = 10;
- - - - - -
 - - - - -
- - - - -
}

Notes

function ( )
{
int count = 0;
- - - - - - -
- - - - - - -
count ++;

}
```

When the function references the variable count, it will be referencing only its local variable, not the global one. The value of count in main( ) will not be affected.

```
/* illustration of working of global variable int x;
illustration of working of global variable
int x;
main( )
{
x = 10;
printf (“x = %d \n”, x);
printf (“x = %d \n”, fun1( ));
printf (“x = %d \n”, fun2( ));
printf (“x = % d \n”, func3( ));
}
fun1( )
{
x = x + 10;
return x;
```

```
}
fun2( )
{
int x = 1;
return x;
}
fun3( )
{
x = x+10;
return (x);
}
Output: x = 10
x = 20
x = 1
x = 30
```

8.4 External Declaration

In the program segment discussed just previously, the main cannot access the variable y as it has been declared after the main function. This problem can be solved by declaring the variable with the storage class extern.

```
main()
{
externint y; /* external declaration */
--------


}
fun1()
{
externint y; /* external declaration */
--------
}
int y; /*definition */
```

The external declaration of y inside the functions informs the compiler that y is an integer type defined somewhere else in the program.

8.5 Static Variable

Static variables are defined within a function in the same manner as automatic variables, except that the variable declaration must begin with the static storage class designation.

static int x; or static float y;

A static variable is initialized only once, when the program is compiled. It is never initialized again.

A static variable may be either an internal type or an external type, depending on the place of declaration. Internal static variables are those which are declared inside a function. The scope of internal static variables extends upto the end of the function in which they are defined. Therefore, internal static variables are similar to auto variables, except that they remain in existence (alive) throughout the remaining program. Therefore, internal static variables can be used to retain values between function calls.

```
/* Illustration of static variable */
main()
{
inti;
for(i=1; i<=3; i++) stat();
}
stat()
{
staticint x = 0;
x = x+1;
printf("x = %d;\t", x);
}
Output: x = 1; x = 2; x = 3
```

An external static variable is declared outside of all functions and is available to all the functions in that program. The difference between a static external variable and a simple external variable is that the static external variable is available only within the file where it is defined while the simple external variable can be accessed by other files also.

8.6 Register Variable

We can tell the compiler that a variable should be kept in one of the machine's registers, instead of keeping in the memory (where normal variables are stored) since a register access is much faster than a memory access and keeping the frequently accessed variables in the register will lead to faster execution of programs.

For example, Loop control variables. This is done as given below:

registerint count;

Since only a few variables can be placed in the register, it is important to carefully select the variables for this purpose. However, C will automatically convert register variables into non-register variables once the limit is reached.

When a function is written before main it can be called in the body of main. If it is written after main then in the declaration of main you have to write the prototype of the function. The prototype can also be written as a global declaration.

Program:

Case 1:

```
#include <stdio.h>
main ( )
{
inti;
```

```
void (int *k) // D
i = 0;
printf (" The value of i before call %d \n", i);
f1 (&i); // A
printf (" The value of i after call %d \n", i);
}
void (int *k) // B
{
*k = *k + 10; // C
}
```

Case 2:

```
#include <stdio.h>
void (int *k) // B
{
*k = *k + 10; // C
}
main ( )
{
inti;
i = 0;
printf (" The value of i before call %d \n", i);
f1 (&i); // A
printf (" The value of i after call %d \n", i);
}
```

Case 3:

```
#include <stdio.h>
void f1(int *k) // B
{
*k = *k + 10; // C
} .
main ( )
{
inti;
i = 0;
printf ("The value of i before call %d \n", i);
f1 (&i); // A
printf ("The value of i after call %d \n", i);
}
```

Explanation

In Case 1, the function is written after main, so you have to write the prototype definition in main as given in statement D.

In Case 2, the function is written above the function main, so during the compilation of main the reference of function f1 is resolved. So it is not necessary to write the prototype definition in main.

In Case 3, the prototype is written as a global declaration. So, during the compilation of main, all the function information is known.

Questions

1. Write a function which receives a float and an int from main ( ), finds the product of these two and returns the product which is printed through main ( ).
2. Write a function that receives 5 integers and returns the sum, average and standard deviation of these numbers. Call this function from main ( ) and print the results in main ( ).
3. Write a function that receives marks received by a student in 3 subjects and returns the average and percentage of these marks. Call this function from main ( ) and print

 Program to demonstrate different storage classes

```
#include <stdio.h>


// declaring the variable which is to be made extern
// an initial value can also be initialized to x
int x;


voidautoStorageClass()
{

        printf("\nDemonstrating auto class\n\n");

        // declaring an auto variable (simply
        // writing "int a=32;" works as well)
        autoint a = 32;

        // printing the auto variable 'a'
        printf("Value of the variable 'a'"
                " declared as auto: %d\n",
                a);

        printf("--------------------------------");
}

voidregisterStorageClass()
{

        printf("\nDemonstrating register class\n\n");

        // declaring a register variable
        register char b = 'G';
```

```
        // printing the register variable 'b'
        printf("Value of the variable 'b'"
                " declared as register: %d\n",
                b);

        printf("--------------------------------");
}

Void extern Storage Class()
{

        printf("\n Demonstrating extern class\n\n");

        // telling the compiler that the variable
        // z is an extern variable and has been
        // defined elsewhere (above the main
        // function)
        externint x;

        // printing the extern variables 'x'
        printf("Value of the variable 'x'"
                " declared as extern: %d\n",
                x);

        // value of extern variable x modified
        x = 2;

        // printing the modified values of
        // extern variables 'x'
        printf("Modified value of the variable 'x'"
                " declared as extern: %d\n",
                x);

        printf("--------------------------------");
}

Void static Storage Class()
{
        inti = 0;
```

```
        printf("\n Demonstrating static class\n\n");

        // using a static variable 'y'
        printf("Declaring 'y' as static inside the loop.\n"
                "But this declaration will occur only"
                " once as 'y' is static.\n"
                "If not, then every time the value of 'y' "
                "will be the declared value 5"
                " as in the case of variable 'p'\n");

        printf("\n Loop started:\n");

        for (i = 1; i< 5; i++) {

                // Declaring the static variable 'y'
                staticint y = 5;

                // Declare a non-static variable 'p'
                int p = 10;

                // Incrementing the value of y and p by 1
                y++;
                p++;

                // printing value of y at each iteration
                printf("\nThe value of 'y', "
                        "declared as static, in %d "
                        "iteration is %d\n",
                        i, y);

                // printing value of p at each iteration
                printf("The value of non-static variable 'p', "
                        "in %d iteration is %d\n",
                        i, p);
        }

        printf("\n Loop ended:\n");

        printf("--------------------------------");
}
```

```
int main()
{

        printf("A program to demonstrate"
                " Storage Classes in C\n\n");

        // To demonstrate auto Storage Class
        Auto Storage Class();

        // To demonstrate register Storage Class
        Register Storage Class();

        // To demonstrate extern Storage Class
        Extern Storage Class();

        // To demonstrate static Storage Class
        Static Storage Class();

        // exiting
        printf("\n\n Storage Classes demonstrated");

        return 0;
}
```

8.7 Const Qualifier

To declare a variable constant, we use the const qualifier. That is, after the variable has been initialized, we cannot modify its value. Const offers a lot of advantages. If you have a constant value for PI, for example, you don't want any element of the programme to change that value. As a result, you should declare it as a const.

The compiler may store objects defined with const-qualified types in read-only memory, and if the address of a const object is never used in a programme, it may not be stored at all.

Summary

- Auto, extern, register, static are the four different storage classes in a C program.
- In this unit, we learnt about "storage classes". A keyword auto is used to define an auto storage class.
- Extern storage class is used when we have global functions or variables which are shared between two or more files
- The static variables are used within function/ file as local static variables. They can also be used as a global variable
- Register is used to store the variable in CPU registers rather memory location for quick access.
- Storage class represents the scope and lifespan of a variable.

- It also tells who can access a variable and from where?

Keywords

Auto	Register
Extern	Static

Self Assessment

1. What is the initial value of register storage class specifier?

A. 0
B. Null
C. Garbage
D. Infinite

2. What is the scope of extern class specifier?

A. Within block
B. Within Program
C. Global Multiple files
D. None of the above

3. What is the scope of static class specifier?

A. Within block
B. Within Program
C. Global Multiple files
D. None of the above

4. Which is not a storage class?

A. Auto
B. Struct
C. Typedef
D. Static

5. What is the output of the following program?

```
#include<stdio.h>

int main()
{
staticint a = 3;
printf("%d", a --);
return 0;
}
```

A. 0
B. 1

C. 2
D. 3

6. What will be the output of the following program?

```
#include <stdio.h>
staticint y = 1;
int main()
{
staticint z;
printf("%d %d", y, z);
return 0;
}
```

A. Garbage value
B. 0 0
C. 1 0
D. 1 1

7. In case of a conflict between the names of a local and global variable what happens?

A. The global variable is given a priority.
B. The local variable is given a priority.
C. Which one will get a priority depends upon which one is defined first.
D. The compiler reports an error.

8. Where will the space be allocated for an automatic storage class variable?

A. In CPU register
B. In memory as well as in CPU register
C. In memory
D. On disk.

9. What is the output of the program?

```
staticint k;
int main()
{
    printf("%d", k);

    return 50;
}
```

A. -1
B. 50
C. 0
D. Compiler error

10. The statement below is a _____?

externint a;

A. Declaration
B. Definition
C. Initialization
D. None of the above

11. An external variable is one-

A. Which is globally accessible by all functions
B. Which is declared outside the body of any function
C. Which resides in the memory till the end of a program
D. All of the above

12. Functions in C are always _____

A. Internal
B. External
C. External and Internal are not valid terms for functions
D. Both Internal and External

13. Global variables are _____

A. External
B. Internal
C. Both internal and external
D. None of above

14. Which storage class is used for faster execution?

A. Register
B. Auto
C. Extern
D. Static

15. Which of the following is default storage class?

A. Register
B. Auto
C. Extern
D. Static

Answers for Self Assessment

1.	C	2.	C	3.	A	4.	B	5.	D
6.	C	7.	B	8.	C	9.	B	10.	A
11.	D	12.	B	13.	A	14.	A	15.	B

Review Questions

1. Write a program to demonstrate static storage classes
2. What is significance of storageclasses.
3. Write a program to demonstrate auto storage classes
4. Write a program to demonstrate extern storage classes
5. Write a program to demonstrate register storage classes

Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

Unit 09: Arrays

CONTENTS

Objectives

- After studying this unit, you will be able to:
- Explain arrays
- Describe two dimensional array
- Describe array initialization

Introduction

An array is a group of data items of same data type that share a common name. Ordinary variables are capable of holding only one value at a time. If we want to store more than one value at a time in a single variable, we use arrays.

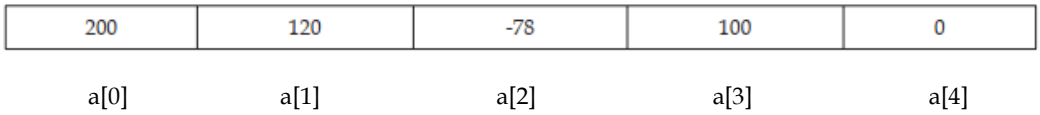
An array is a collective name given to a group of similar quantities. Each member in the group is referred to by its position in the group.

Arrays are allotted the memory in a strictly contiguous fashion. The simplest array is one dimensional array which is simply a list of variables of same data type. An array of one dimensional arrays is called a two dimension array.

9.1 Arrays

Arrays are allocated the memory in a strictly contiguous fashion. The simplest array is one dimensional array which is a list of variables of same data type. An array of one dimensional arrays is called a two dimensional array; array of two dimensional arrays is three dimensional array and so on.

The members of the array can be accessed using positive integer values (indicating their order in the array) called subscript or index. Look at an array of integers as shown below:



The description of this array is listed below:

Name of the array : a

Data type of the array : integer

Number of elements : 5

Valid index values : 0, 1, 2, 3, 4

Value stored at the location a[0] : 200

Value stored at the location a[1] : 120

Value stored at the location a[2] : -78

Value stored at the location a[3] : 100

Value stored at the location a[4] : 0

9.2 Advantages of Arrays

Arrays offer a number of advantages, some of which are elucidated below:

1. If only a limited number of variables of a particular data type is required ion a program, one can choose the variable names to suite the situation. Let us say we require five integer type variables, we can define them as follows:

int v_one, v_two, v_three, v_four, v_five;

Now, consider if we require hundred integer type variables, is the above approach convenient? Obviously not. We can, instead, use an array of integer type having 100 elements as shown below:

int num[100];

2. Array elements can be accessed using index. Therefore, all the elements can be processed in a desired manner in a single for loop that runs for each element, as shown below:

for(i=0; i<100; i++)

num[i]=num[i]+10;

In a single for loop, all the elements have been incremented by 10.

3. Since array elements are physically created contiguously in the memory, they can be accesses using pointers (as you will learn later). Therefore, there are more than one way to reference array elements.

9.3 Types of Arrays

According the number of subscripts required to access an array element, arrays can be of

Following types:

1. One-dimensional array

2. Multi-dimensional array

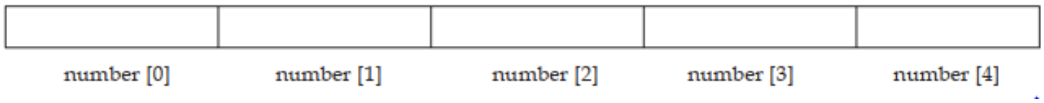
9.4 One-dimensional Array

A list of items can be given one variable name using only one subscript and such a variable is called a one dimensional array.

Example: If we want to store a set of five numbers by an array variable number. Then it will be accomplished in the following way:

int number [5];

This declaration will reserve five contiguous memory locations capable of storing an integer type value each, as shown below:



As C performs no bounds checking, care should be taken to ensure that the array indices are within the declared limits. Also, indexing in C begins from 0 and not from 1.

9.5 Two-dimensional and Multi-dimensional Array

It is possible to have an array of more than one dimensions. Two dimensional array (2-D array) is an array of number of 1-dimensional arrays.

A two dimensional array is also called a matrix. Consider the following table:

	Item1	Item2	Item3
Sales 1	300	275	365
Sales 2	210	190	325
Sales 3	405	235	240
Sales 4	260	300	380

This is a table of four rows and three columns. Such a table of items can be defined using two dimensional arrays.

General form of declaring a 2-D array is

data_type array_name [row_size] [colum_size];

Example: int marks [4] [2];

It will declare an integer array marks of four rows and two columns. An element of this array can be accessed by the manipulation of both the indices. printf ("%d", marks [2] [1]) will print the element present in third row and second column.

C allows arrays of three or more dimensions. Multi-dimensional arrays are defined in much the same manner as one-dimensional arrays, except that a separate pair of square brackets is required for each subscript.

The general form of a multi-dimensional array is

data_type array_name [s1] [s2] [s3] . . . [sm];

E.g.: int survey [3] [5] [12];

float table [5] [4] [5] [3];

Here, survey is a 3-dimensional array declared to contain 180 integer type elements. Similarly, table is a 4-dimensional array containing 300 elements of floating point type.

Let us consider some applications of multidimensional array programming.

- Sorting an integer array.

```
# include <stdio.h>

void main( )

{

int arr [5];

int i, j; temp;

printf ("\n Enter the elements of the array:"};
```

```
scanf ("%d", & arr [i]);
for (i = 0; i < = 4; i ++);
{
for (J = 0; J < = 3; J ++)
if (arr [J] > arr [J+1])
{
temp = arr [J];
arg [J] = arr [J+1];
arr [J+1] = temp;
}
}
printf ("\ n The Sorted array is:");
for (i = 0; i < 5; i++)
printf ("\ t %d", arr [i]);
}
```

- To insert an element into an existing sorted array (Insertion Sort).

```
# include <stdio.h>
main( )
{
int i, k, y, x [20], n;
for (i = 0; i < 20; i++)
x [ i] = 0;
printf ("\ Enter the number of items to be inserted:\n");
scanf ("%d", &n);
printf ("\n Input %d values \n", n);
for (k = 0; k < n; k++)
{
scanf ("%d", &x [k]);
y = x [x]
for (i = k-1; i > = 0 && y < x [i]; i - -)
x [i+1] = x[i];
x [i+1] = y;
}
printf ("\n The sorted numbers are:");
for (i = 0; i < n; i++)
printf ("\n %d", x [i]);
}
```

- Accept character string and find its length.

We will solve this question by looping instead of using Library function strlen( ).

```
# include <stdio.h>
void main( )
{
char name [20];
int i, len;
printf ("\n Enter the name:");
scanf ("%s", name);
for (i = 0; name [i] ! = '\0'; i++);
Len = i - 1;
print f("\n Length of array is % d", len);
```

Character Arrays

Just as a group of integers can be stored in an integer array, group of characters can be stored in a character array or "strings". The string constant is a one dimensional array of characters terminated by null character ('\0'). This null character '\0' (ASCII value0) is different from 'O'

(ASCII value 48).

The terminating null character is important because it is the only way the function that works with string can know where the string ends.

Example: Static char name [ ] = {'K', 'R', 'I', 'S', 'H', '\0'};

This example shows the declaration and initialization of a character array. The array elements of a character array are stored in contiguous locations with each element occupying one byte of memory.

K	R	I	S	H	N	A	'\0'
4001	4002	4003	4004	4005	4006	4007	4009

1. Contrary to the numeric array where a 5 digit number can be stored in one array cell, in the character arrays only a single character can be stored in one cell. So in order to store an array of strings, a 2-dimensional array is required.
2. As scanf( ) function is not capable of receiving multi word string, such strings should be entered using gets( ).

Point out the errors, if any, in this program:

```
main(){

    int i, a - 2, b - 3 ;
    int arr.[ 2 +3] ;
```

```
for (i 0;i< a+b;i++ )
{
        scanf ( "%d", &rarr[i] ) ;
        printf ( " \ n%d", arr[i] ) ;
}
}
```

9.6 Array Declaration

Arrays are defined in the same manner as ordinary variables, except that each array name must be accompanied by the size specification.

The general form of array declaration is:

data_type array_name [size];

data-type specifies the type of array, size is a positive integer number or symbolic constant that indicates the maximum number of elements that can be stored in the array.

: float height [50];

This declaration declares an array named height containing 50 elements of type float. The compiler will interpret first element as height [0]. As in C, the array elements are induced

for 0 to [size-1].

Two dimensional arrays can be declared similarly, as shown below:

data_type array_name[size1][size2];

For instance, the following array (named b) is array of 2 arrays of integer type of size 5

elements:

int b[2][5];

The array b has 10 (2 * 5) elements, each capable of storing an integer type data, referenced as:

b[0][0] b[0][1] b[0][2] b[0][3] b[0][4]

b[1][0] b[1][1] b[1][2] b[1][3] b[1][4]

Multidimensional arrays can be declared on the similar lines. A three dimensional array (named

c) of int type has been declared below:

Int c[2][2][5];

The array c has 20 (2 * 2 * 5) elements, each capable of storing an integer type data, referenced

as:

c[0][0][0] c[0][0][1] c[0][0][2] c[0][0][3] c[0][0][4]

c[0][1][0] c[0][1][1] c[0][1][2] c[0][1][3] c[0][1][4]

c[1][0][0] c[1][0][1] c[1][0][2] c[1][0][3] c[1][0][4]

c[1][1][0] c[1][1][1] c[1][1][2] c[1][1][3] c[1][1][4]

9.7 Array Initialization

One-dimensional Array

The elements of an array can be initialized in the same way as the ordinary variables, when they are declared. Given below are some examples which show how the arrays are initialized.

static int num [6] = {2, 4, 5, 45, 12};

static int n [ ] = {2, 4, 5, 45, 12};

static float press [ ] = {12.5, 32.4, -23.7, -11.3};

In these examples note the following points:

1. Till the array elements are not given any specific values, they contain garbage value.

2. If the array is initialized where it is declared, its storage class must be either static or extern.

If the storage class is static, all the elements are initialized by 0.

3. If the array is initialized where it is declared, mentioning the dimension of the array is optional.

Two-dimensional Arrays

Two dimensional arrays may be initialized by a list of initial values enclosed in braces following their declaration.

E.g.: static int table[2][3] = {0, 0, 0, 1, 1, 1};

initializes the elements of the first row to 0 and the second row to one. The initialization is done by row.

The aforesaid statement can be equivalently written as

static int table[2][3] = {{0, 0, 0}, {1, 1, 1}};

by surrounding the elements of each row by braces.

We can also initialize a two dimensional array in the form of a matrix as shown below:

static int table[2][3] = {{0, 0, 0},

{1, 1, 1}};

The syntax of the above statement. Commas are required after each brace that closes off a row, except in the case of the last row.

If the values are missing in an initializer, they are automatically set to 0. For instance, the statement

static int table [2] [3] = {{1, 1},

{2}};

will initialize the first two elements of the first row to one, the first element of the second row to two, and all the other elements to 0.

When all the elements are to be initialized to 0, the following short cut method may be used.

static int m [3] [5] = {{0}, {0}, {0}};

The first element of each row is explicitly initialized to 0 while other elements are automatically initialized to 0.

While initializing an array, it is necessary to mention the second (column) dimension, whereas the first dimension (row) is optional. Thus, the following declarations are acceptable.

static int arr [2] [3] = {12, 34, 23, 45, 56, 45};

static int arr [ ] [3] = {12, 34, 23, 45, 56, 45 };

Multi-dimensional Array

Example: Example of initializing a 4-dimensional array:

static int arr [3] [4] [2] = {{{2, 4}, {7, 8}, {3, 4}, {5, 6},},

{{7, 6}, {3, 4}, {5, 3}, {2, 3}, },

{{8, 9}, {7, 2}, {3, 4}, {6, 1}, } };

In this example, the outer array has three elements, each of which is a two dimensional array of

four rows, each of which is a one dimensional array of two elements.

9.8 Accessing Elements of an Array

Once an array is declared, individual elements of the array are referred using subscript or index number. This number specifies the element's position in the array. All the elements of the array are numbered starting from 0. Thus number [5] is actually the sixth element of an array.

Consider the program given above. It has entered 6 values in the array num. Now to read values from this array, we will again use for Loop to access each cell. The given program segment explains the retrieval of the values from the array.

```
for (count = 0; count < 6; count ++)
{
printf ("\n %d value =", num [count]);
}
```

Data can be inserted into array by treating the array elements just like any other variable. If an integer value is to be read from keyboard into an array element (say c[2][3][0]), the following code snippet would do the job:

```
Scanf("%d", &c[2][3][0]);
```

In order to read values in the entire array for loop may be used as explained by the following examples:

```
main( )
{
int num [6];
int count;
for (count = 0; count < 6; count ++)
{
printf ("\n Enter %d element:" count+1);
scanf ("%d", &num [count]);
}
}
```

In this example, using the for loop, the process of asking and receiving the marks is accomplished. When count has the value zero, the scanf( ) statement will cause the value to be stored at num [0].

This process continues until count has the value greater than 5.


Case Study

Each element of the array has a memory address. The following program prints an array limit value and an array element address.

Program:

```
#include <stdio.h>
void printarr(int a[]);
main()
{
int a[5];
for(int i = 0;i<5;i++)
{
a[i]=i;
}
printarr(a);
}
void printarr(int a[])
{
```

```
for(int i = 0;i<5;i++)
{
printf("value in array %d\n",a[i]);
}
}
void printdetail(int a[])
{
for(int i = 0;i<5;i++)
{
printf("value in array %d and address is %16lu\n",a[i],&a[i]);
\\ A
}
}
```

Explanation

1. The function printarr prints the value of each element in arr.

2. The function printdetail prints the value and address of each element as given in statement A. Since each element is of the integer type, the difference between addresses is 2.

3. Each array element occupies consecutive memory locations.

4. You can print addresses using place holders %16lu or %p.

Questions

1. Write a program to add two 6 x 6 matrices.

2. Write a program to multiply any two 3 x 3 matrices.

3. Write a program to sort all the elements of a 4 x 4 matrix.

4. Write a program to obtain the determinant value of a 5 x 5 matrix.

9.9 Passing array as an argument to function

If you want to pass a single-dimension array as an argument in a function, you would have to declare a formal parameter in one of following three ways and all three declaration methods produce similar results because each tells the compiler that an integer pointer is going to be received. Similarly, you can pass multi-dimensional arrays as formal parameters.

Method -1

Formal parameters as a pointer −

```
void myFunction(int *param) {
  .
  .
  .
}
```

Method -2

Formal parameters as a sized array −

```
void myFunction(int param[10]) {
  .
```

```
  .
  .
}
```

Method -3

Formal parameters as an unsized array −

```
void myFunction(int param[]) {
  .
  .
  .
}
```

 **:** Now, consider the following function, which takes an array as an argument along with another argument and based on the passed arguments, it returns the average of the numbers passed through the array as follows −

```
double getAverage(int arr[], int size)
{
  int i;
  double avg;
  double sum = 0;

  for (i = 0; i < size; ++i) {
    sum += arr[i];
  }
  avg = sum / size;
  return avg;
}
```

Now, let us call the above function as follows −

```
#include <stdio.h>
 double getAverage(int arr[], int size);
int main () {
  int balance[5] = {1000, 2, 3, 17, 50};
  double avg;
  avg = getAverage( balance, 5 ) ;
  printf( "Average value is: %f ", avg );
     return 0;
}
```

Summary

- An array is a group of memory locations related by the fact that they all have the same name and same data type.
- An array including more than one dimension is called a multidimensional array.
- The size of an array should be a positive number. If an array in declared without a size and in initialized to a series of values it is implicitly given the size of number of initializers.
- Array subscript always starts with 0. Last element's subscript is always one less than the size of the array e.g., an array with 10 elements contains element 0 to 9. Size of an array must be a constant number.

Keywords

Array: A user defined simple data structure which represents a group of same type of variables having same name each being referred to by an integral index

Multidimensional array: An array in which elements are accessed using multiple indices

One dimensional array: An array in which elements are accessed using a single index

Subscript/Index: The integral index by which an array element is accessed

Two dimensional array: An array in which elements are accessed using two indices

Self Assessment

1. What is an Array in C language?

A. A group of elements of same data type.

B. An array contains more than one element.

C. Array elements are stored in memory in continuous or contiguous locations.

D. All the above.

2. An array Index starts with?

A. 1

B. 0

C. -1

D. 2

3. Arrays can

A. store data elements of same data type at contiguous memory location.

B. be used for CPU scheduling.

C. be used for reverse data elements, sort data elements etc.

D. All of above

4. Two dimensional arrays in C

A. An array of arrays is known as two dimensional array.

B. An array of loops

C. An array of tokens

D. All of above

5. Choose the correct syntax for two dimensional array

A. data_type name_of_array;

B. data_type name_of_array[rows][columns];

C. data_type [rows][columns];

D. name_of_array[rows][columns];

6. What will be the output of the following C code?

```
#include <stdio.h>
  void main()
  {
     int a[2][3] = {1, 2, 3, 4, 5};
     int i = 0, j = 0;
     for (i = 0; i < 2; i++)
     for (j = 0; j < 3; j++)
     printf("%d", a[i][j]);
  }
```

A. 1 2 3 4 5 0

B. 1 2 3 4 5 junk

C. 1 2 3 4 5 5

D. Run time error

7. One-dimensional array of two-dimensional arrays is called

A. One-dimensional array

B. Multi-dimensional array

C. Two-dimensional array

D. Three-dimensional array

8. How many kinds of elements an array can have?

A. Char and int type
B. Only char type
C. Only int type
D. All of them have same type

9. Choose the correct statement
A. Array stores data of the same type
B. Array can be a part of a structure
C. Array of structure is allowed
D. All of the above

10. Choose correct declaration of function with as array parameter, let name of function is sum and array (arr) is of integer type
A. int sum(int)
B. int sum(int arr[])
C. int sum(arr[])
D. int sum(arr)

11. Choose the right statement
A. Arrays and function can't be used in C language.
B. Passing functions to array is possible in C Language
C. Passing array to function is possible in C Language
D. All of above

12. A character constant is enclosed by?
A. Left Single Quotes
B. Right Single Quotes
C. Double Quotes
D. None of the above

13. A character array can be initialized using
A. Floats value
B. Integer values
C. A string literal
D. None of them

14. Array is a group of data items of

A. Same data type that share a common name

B. Same data type that share a uncommon name

C. Not data type that never common name

D. None of the above

15. The general form of array declaration is

A. array_name [size];

B. data_type array_name [size];

C. data_type [size];

D. None

Answers for Self Assessment

1.	D	2.	B	3.	D	4.	A	5.	B
6.	A	7.	C	8.	D	9.	D	10.	B
11.	C	12.	B	13.	C	14.	A	15.	B

Review Questions

1. Explain the usefulness of Arrays in C.
2. What do you mean by 'Array'? How it can be declared & initialized in a C program?
3. Draw a diagram to represent the internal storage of an Array.
4. Describe the different types of Array. Give suitable programs.
5. Find the smallest number in an array using pointers.
6. If an array arr contains n elements, then write a program to check if arr[0] = arr[n-1], arr[1] = arr[n-2] and so on.
7. Write a program to copy the contents of one array into another in the reverse order.
8. How will you initialize a three-dimensional array threed[3][2][3]? How will you refer the first and last element in this array?
9. Write a program to pick up the largest number from any 5 row by 5 column matrix.
10. Write a program to obtain transpose of a 4 x 4 matrix. The transpose of a matrix is obtained by exchanging the elements of each row with the elements of the corresponding column.
11. Write a program that interchanges the odd and even components of an array.

Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India,

New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C

Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com